Movie Recommendation System
Design Document

Authors:
Ibrahim Awan
Jacky Zheng
Omar AlShantout
Hridi Paul

Group: 65

This page intentionally left blank.

Document Revision History

Date	Version	Description	Author
11/17/2025	0.0	Initial draft	Ibrahim Awan Jacky Zheng Omar AlShantout Hridi Paul

This page intentionally left blank.

Contents

1 Introduction 7

1.1 Purpose 7

1.2 System Overview 7

1.3 Design Objectives 7

1.3.1 Design Goals of the System and This Document 7
1.3.2 Functionalities of Stakeholders Covered in This Document 7
1.3.3 Non-Functional Requirements of the System 8
1.3.4 Requirements of This Document 8

1.4 References 8

1.5 Definitions, Acronyms, and Abbreviations 8

2 Design Overview 8

2.1 Introduction 8

2.2 Environment Overview 9

2.3 System Architecture 9

2.3.1 User Interface Layer 10
2.3.2 Business Logic (BI) Layer 10
2.3.3 System Support Layer 10

2.4 Constraints and Assumptions 10

3 Interfaces and Data Stores 10

3.1 System Interfaces 10

3.1.1 Command-Line User Interface 10
3.1.2 Application Service Interfaces 11
3.1.3 File System Interface 11

3.2 Data Stores 11

3.2.1 Persistent User Rating Data – my_ratings.txt 11
3.2.2 In-Memory Stores 12

4 Structural Design 13

4.1 Class Diagram 13

4.2 Class Descriptions 14

4.2.1 Class: ConsoleUI 14
4.2.2 Class: RatingManager 15
4.2.3 Class: FileHandler 16
4.2.4 Class: RecommendationEngine 18
4.2.5 Class: Movie 19

5 Dynamic Model 20

5.1 Rating a Movie Scenario 20

5.2 Generating Movie Recommendations Scenario 21

6 Error Handling Strategy 21

6.1 Principles 21

6.2 Error Cases and Results 21

6.2.1 Inability to Start....21
6.2.2 Recoverable File....22
6.2.3 Parsing....22
6.2.4 User Input....22
6.2.5 Algorithmic Edge....22
6.3 Responsibilities per layer....22
6.3.1 ConsoleUI....22
6.3.2 Business logic (RatingManager, RecommendationEngine)....22
6.3.3 FileHandler....22
6.4 Standard Messages from CLI....22
6.4.1 Startup Inability....22
6.4.2 Invalid Menu Choice....23
6.4.3 Invalid Movie ID....23
6.4.4 Invalid Rating....23
6.4.5 Write Failure....23
6.5 Error Scenarios....23
6.5.1 movies.txt invalid....23
6.5.2 my_ratings.txt missing....23
6.5.3 Invalid lines in the catalog....23
6.5.4 Invalid user input....23
6.6 Methods of testing....23
7 Non-functional Requirements....23
7.1 Performance Requirements....23
7.2 Safety Requirements....24
7.3 Security Requirements....24
7.4 Usability Requirements....24
7.5 Maintainability Requirements....25
7.6 Reliability Requirements....25
7.7 Scalability Requirements....25
7.8 Portability Requirements....25
7.9 Future Evolution....26

1 Introduction

1.1 Purpose

This document describes the software design for the Movie Recommendation System developed for COMPSCI 2ME3. Its purpose is to give developers, instructors, and future project contributors a clear and organized description of the system's architecture and internal class structure. Although Milestone 1 emphasizes the Waterfall methodology and does not require a finished implementation, this document outlines how the system is intended to be built so that Milestone 2 may proceed smoothly with implementation. It includes the architectural overview in Section 2, detailed module and data structure descriptions in Section 3, UML diagrams and class specifications in Section 4, dynamic behavior models in Section 5, and the non-functional design considerations in Section 6.

1.2 System Overview

The Movie Recommendation System is a Java application that loads movie data, reads or creates a user ratings file, and produces movie recommendations using content-based filtering. It allows the user to browse available movie information, add new ratings, update existing ratings, and receive recommendations based on genres associated with films they rated highly. The system relies on two CSV files: movies.txt, which stores the master catalog of movies, and my_ratings.txt, which stores user-generated rating data. Milestone 1 focuses primarily on the analysis, design, and documentation of the system, and only partial implementation. Please review the Software Requirement Spec for the full set of requirements for this project.

1.3 Design Objectives

1.3.1 Design Goals of the System and This Document

The Movie Recommendation System is designed to follow modular software architecture, where responsibilities are cleanly separated among components for file parsing, recommendation logic, rating management, and user interaction. Interactions between components should be minimal to ensure maintainability and ease of extension in Milestone 2. This document provides a complete and structured design including the architecture, interfaces, class diagrams, and core algorithms, ensuring that developers have all required information before implementation.

1.3.2 Functionalities of Stakeholders Covered in This Document

The main user of this system is the movie viewer, who will perform tasks such as viewing movies from the catalog, assigning ratings, updating previous ratings, and receiving personalized recommendations. Developers are also key stakeholders, and this document provides them with the detailed design required to guide implementation in Milestone 2. Instructors and teaching assistants use the document to verify that the design aligns with the functional and non-functional requirements outlined in the SRS. Future team members working in Milestone 2 will also rely on this design to extend or complete the system without needing to revisit the requirements phase.

1.3.3 Non-Functional Requirements of the System

The system must be robust enough to handle malformed or missing data in both CSV files without crashing, logging errors when necessary and continuing operation safely. Its architecture must be maintainable so that future enhancements in Milestone 2 can be integrated without extensive restructuring. The system should also be easy to use, offering a simple and intuitive interaction flow whether users run the application through a command-line interface or a basic GUI that may be implemented in a future release. Reliability and safety are essential, particularly for ensuring that user rating data is not lost or corrupted during file operations. Performance is also essential as the system must be responsive and capable of generating recommendations and rating movies quickly. Again, see the SRS for more extensive listing of non-functional requirements.

1.3.4 Requirements of This Document

This document must present all necessary structural, behavioral, and architectural details so developers can implement the system without needing additional information. It must remain consistent with the SRS by accurately reflecting its requirements and ensuring all design components can be traced back to their corresponding specifications.

1.4 References

This document is based on the COMPSCI 2ME3 Milestone #1: Waterfall Process assignment written by Dr. Irene Ye Yuan, which outlines the expectations for requirements analysis and design. It may also reference the Movie Recommendation System SRS created for this milestone, which defines all functional and non-functional requirements. Additional references include the Java SE 17 documentation and the Java I/O API documentation for file operations. Course templates for the SDD provided on Avenue are also used to guide the structure and content of this document.

1.5 Definitions, Acronyms, and Abbreviations

The Movie Recommendation System refers to the software project developed during Milestones 1 and 2. A CLI is a Command-Line Interface that may be used to allow users to interact with the system. SRS stands for Software Requirements Specification, while SDD refers to this Software Design Document. CSV is the comma-separated value format used for movies.txt and my_ratings.txt. Content-based filtering describes the method used to generate recommendations by comparing movie attributes, primarily genres. Lastly, the Movie Catalog refers to the data contained in movies.txt.

2 Design Overview

2.1 Introduction

An object-oriented design approach was adopted for this project. For example, the FileHandler class encapsulates all functionality related to reading and writing the relevant text files. Both RatingManager and RecommendationEngine use the same instance of FileHandler, allowing them to access consistent file-handling behavior through its public methods. The proposed architecture for the system is a layered architecture to promote

separation of business logic, console printing, and file handling. The primary tool used for drawing software modeling diagrams is the Notability application.

2.2 Environment Overview

The application runs on any desktop or laptop system capable of executing Java SE 17 or later. It is compatible with Windows 10/11, macOS, and major Linux distributions that support a Java runtime environment (JRE) of version 17 or later. The system requires minimal hardware resources and functions entirely offline, reading and writing from local files. No network connectivity or external dependencies are required. To run the application, users must have the deployed .jar file on their machine, along with the movies.txt and my_ratings.txt files located in the appropriate directory relative to the application. Additional environment details and setup instructions will be documented in the project's README.md file as development progresses.

2.3 System Architecture

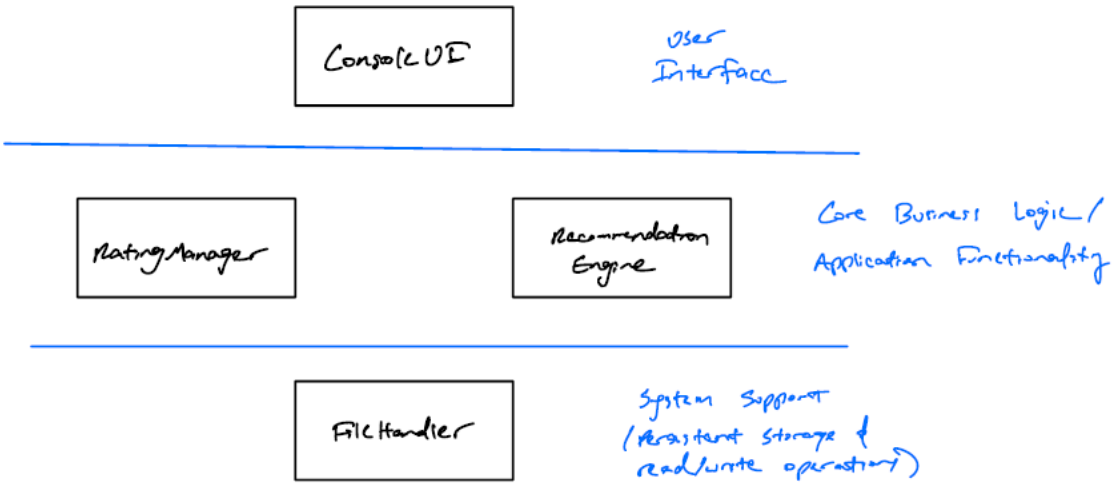


Figure 1: Layered Architecture of the Movie Recommendation System

Figure 1 above shows the layered architecture of the Movie Recommendation System. The advantage of this type of software architecture is that each layer relies on the services offered by the layer immediately beneath it. This is why, as shown in the UML class diagram in Section 4, the ConsoleUI class has private fields for both RatingManager and RecommendationEngine and both RatingManager and RecommendationEngine hold a private field for FileHandler. Because the layers are separate and independent, replacing the current terminal-based user interaction with a graphical user interface in the future would require changes only to the User Interface layer. A clear description of relationships between different components and the responsibility of each component is listed below:

2.3.1 User Interface Layer

This component is to handle the logic for prompting user for input via printing available user options to the terminal and triggering business logic based on user feedback. If something unexpected occurs during program execution, this component is also responsible for printing error messages to the terminal for the user. Because this component must invoke business logic, the relevant class for this component (ConsoleUI) must interact with the BI layer and therefore, holds references to both RatingManager and RecommendationEngine.

2.3.2 Business Logic (BI) Layer

This component is responsible for executing the application functionality (see Sections 4.2 and 4.3 of SRS document for details). However, system features rely heavily on data stored within text files that need to be read into the internal data structures of the program. Due to this, both classes within this layer hold a reference to the FileHandler class.

2.3.3 System Support Layer

This component is responsible for handling all read/write operations that support the BI layer. The class for this layer, FileHandler, provides functionality for fetching movie catalog and user rating data as well as persistently writing new user ratings to my_ratings.txt.

2.4 Constraints and Assumptions

There were no major design constraints imposed by external parties and therefore, the architectural design was chosen based on the project's technical requirements and the development team's judgment.

3 Interfaces and Data Stores

3.1 System Interfaces

3.1.1 Command-Line User Interface

Purpose: Provides user-facing interface for invoking features and viewing results.

Capabilities:

- Display title banner, main menu (see Section 3.1.3 of SRS doc), and prompts
- Accept movie id and rating user input
- Render movie lists and recommendation results
- Report application errors without terminating the app

Inputs: Keyed inputs
Outputs: Plain UTF-8 text. Single-line errors with *'ERROR:'* and confirmations with '*OK:'*. Preconditions: Application started in a terminal with access to the working directory.
Postconditions: On 'q' confirmation, program terminates.

Failure modes: Invalid input requires re-prompting the user and clearly displaying error messages.
Startup Precondition: movies.txt must exist in the required path and contain at least 1 valid movie record

3.1.2 Application Service Interfaces

1) ConsoleUI -> RatingManager
    a. Operation: rateMovie (id: int, rating: int): void
        Effect: Creates/updates user's rating for a movie
2) ConsoleUI -> RecommendationEngine
    a. Operation: getMovieCatalog(): List<Movie>
        Effect: Returns all movies for display/search
    b. Operation: generateMovieRecommendations(): List<Movie>
        Effect: Returns recommended movies based on user ratings
3) RatingManager/RecommendationEngine -> FileHandler.
    a. (Read) Operation: fetchMovieCatelog(): List<Movie>
        Effect: Returns the entire movie catalog
    b. (Read) Operation: fetchUserRatings(): Map<Integer, Integer>
        Effect: Returns a map containing each movie rated by the user and the associated rating value
    c. (Write) Operation: updateExistingRating(id: int, rating: int, date: DateTime): void
        Effect: Updates existing movie rating
    d. (Write) Operation: addNewRating(id: int, rating: int, date: DateTime): void
        Effect: Writes new movie rating

3.1.3 File System Interface

Paths: All reads/writes confined /data directory of the application.
Permissions: Requires read access */data/movies.txt* and read/write access to */data/my_ratings.txt;* ~ 1 mb of disk space.
Clock: Use system clock for *YYYY-MM-DD* timestamps.
Failure Modes: Missing files, permissions, or disk-full results in printing 'ERROR*: (error msg)'* and returns to the menu or exits.

3.2 Data Stores

3.2.1 Persistent User Rating Data – my_ratings.txt

Type: CSV, one line per movie rating.
Schema: *movie_id:int, rating:int (1-5), date: YYYY-MM-DD*
Creation: Created on the first successful rating if absent.
Uniqueness: One row per *movie_id;* upon a rerating, the existing row and date will be overwritten.

Validation: *movie_id* must exist, rating must be between 1 to 5, date will be from system clock.

3.2.2 In-Memory Stores

List<Movie> movieCatalog
Set<Integer> movieCatalogIds
Set<Integer> ratedMovieIds
…

See Section 4 for information regarding the usage of these in-memory data stores.

4 Structural Design

4.1 Class Diagram

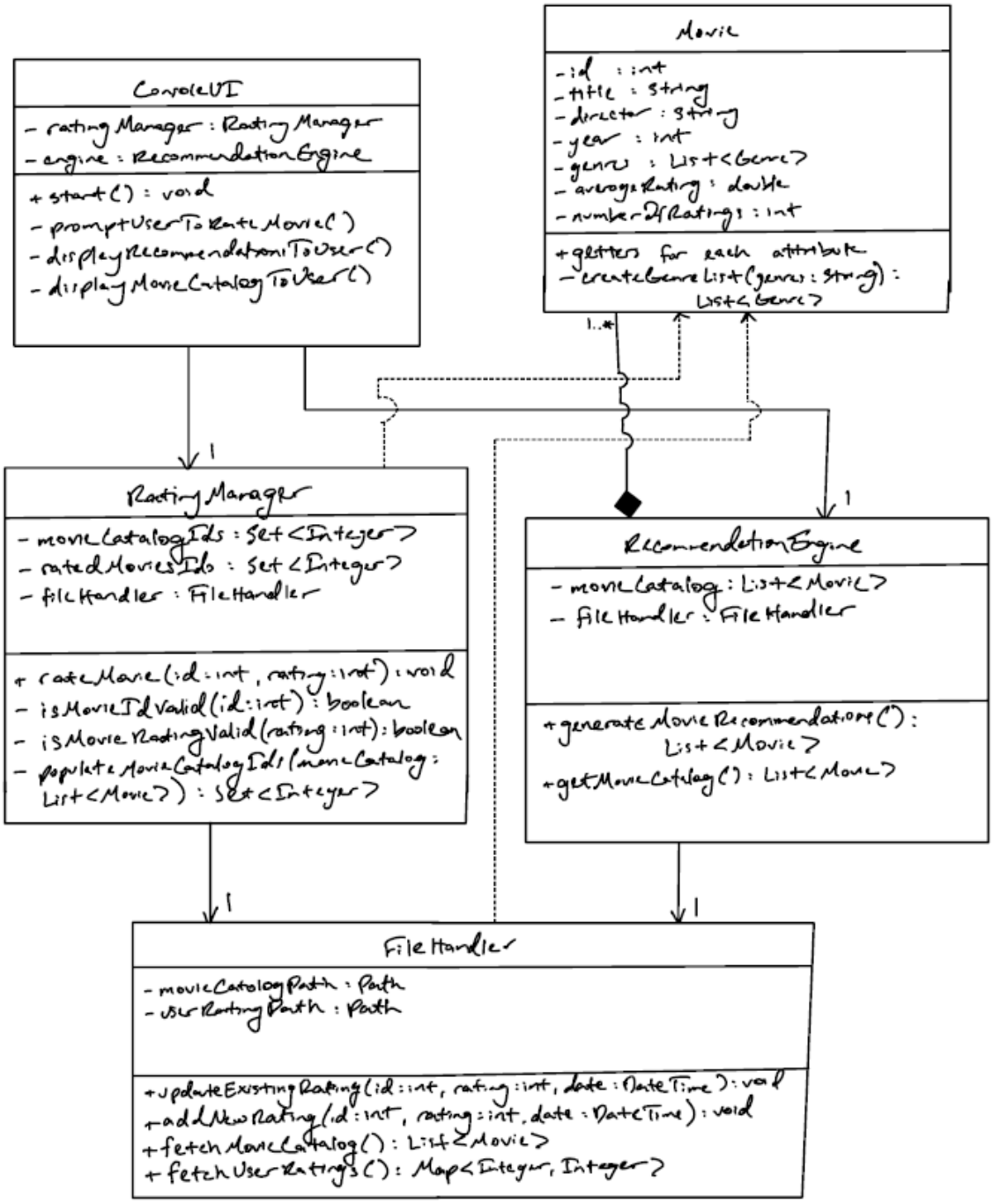


Figure 2: UML Class Diagram of the Movie Recommendation System

4.2 Class Descriptions

4.2.1 Class: ConsoleUI

- Purpose: Trigger business logic depending on user input, and display information to the terminal relevant to the user

4.2.1.1 Attribute Descriptions

1. Attribute: *ratingManager*
   Type: RatingManager
   Description: *Manages user rating logic that is triggered when user wants to rate a movie*
   Constraints*: non-null*
2. Attribute: *engine*
   Type: RecommendationEngine
   Description: *Manages movie recommendation logic that is triggered when user wants movie recommendations*
   Constraints*: non-null*

4.2.1.2 Method Descriptions

1. Method: *start()*
   Return Type: *void*
   Parameters: *N/A*
   Return value: *N/A*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *None*
   Methods called: *promptUserToRateMovie(), displayRecommendationsToUser(), displayExistingMoviesToUser()*

   Processing logic:
   This method includes the while-loop that will continue to prompt the user to either rate a movie, generate recommendations, display movie catalog, or exit the application. The application will end once the user explicitly enters "q" (quit).

2. Method: *promptUserToRateMovie()*
   Return Type: *void*
   Parameters: *N/A*
   Return value: *N/A*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *ratingManager*
   Methods called: *ratingManager.rateMovie()*

   Processing logic:
   This method triggers the business logic for rating a movie. The method first asks the user to provide a movie id and a rating and then calls ratingManager.rateMovie() with the user-provided information.

3. Method: *displayRecommendationsToUser()*
   Return Type: *void*
   Parameters: *N/A*
   Return value: *N/A*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *engine*

Methods called: *engine.generateMovieRecommendations()*

Processing logic:
This method triggers the business logic for generating movie recommendations. The method prints the list of movies returned from engine.generateMovieRecommendations() in a tabular format for the user.

4. Method: *displayMovieCatalogToUser()*
Return Type: *void*
Parameters: *N/A*
Return value: *N/A*
Pre-condition: *N/A*
Post-condition: *N/A*
Attributes read/used: *engine*
Methods called: *engine.getMovieCatalog()*

Processing logic:
This method simply prints the list of movies returned from engine.getMovieCatalog() in a tabular format for the user.

4.2.2 Class: RatingManager

- Purpose: Contains the business logic for allowing users to rate movies and update existing ratings with persistence

4.2.2.1 Attribute Descriptions

1. Attribute: *movieCatalogIds*
Type: Set<Integer>
Description: *Stores all possible movie id's available for a user to rate*
Constraints*: Initialized to a non-zero size during object instantiation. Unchanged state after initialization.*
2. Attribute: *ratedMovieIds*
Type: Set<Integer>
Description: *Stores movie id's of rated movies*
Constraints*: Initialized to zero size during object instantiation but grows as the user continues to rate movies.*
3. Attribute: *fileHandler*
Type: FileHandler
Description: *Provides access to file handling functionality to initialize the movieCatalogIds field and call the appropriate methods responsible for writing ratings to my_ratings.txt.*
Constraints*: non-null*

4.2.2.2 Method Descriptions

1. Method: *isMovieIdValid(int id)*
Return Type: *boolean*
Parameters: *id – the movie id that the user wants to rate*
Return value: *true (valid id), false (invalid id)*
Pre-condition: *N/A*
Post-condition: *N/A*
Attributes read/used: *movieCatalogIds*
Methods called: *N/A*

Processing logic:
Checks if the parameter id exists within movieCatalogIds. If it does, return true. Otherwise, return false.

2. Method: *isMovieRatingValid(int rating)*
   Return Type: *boolean*
   Parameters: *rating – the rating that the user wants to assign a movie*
   Return value: *true (valid rating), false (invalid rating)*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *N/A*
   Methods called: *N/A*

   Processing logic:
   Checks if the parameter rating is an integer between 1-5 (inclusive). If it is, return true. Otherwise, return false.

3. Method: *populateMovieCatalogIds(List<Movie> movieCatalog)*
   Return Type: *Set<Integer>*
   Parameters: *movieCatalog - list of all movies and their associated data read from movies.txt*
   Return value: *All movie id's available for a user to rate*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *movieCatalogIds*
   Methods called: *movie.getId()*

   Processing logic:
   Used to initialize movieCatalogIds field in the RatingManager constructor. This method receives as input all movie data and populates the field with only the movie id's.

4. Method: *rateMovie(int id, int rating)*
   Return Type: *void*
   Parameters: *id – user provided movie id; rating – user provided movie rating*
   Return value: *N/A*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *fileHandler, ratedMovieIds*
   Methods called: *isMovieIdValid(), isMovieRatingValid(), fileHandler.updateExistingRating(), fileHandler.addNewRating()*

   Processing logic:
   This method calls the movie and rating validation private methods to verify user input. Afterwards, it checks if this movie has already been rated by the user in the past. If so, it calls fileHandler.updateExistingRating(). If this movie is being rated for the first time, it calls fileHandler.addNewRating() and adds the movie id in the ratedMovieIds field.

4.2.3 Class: FileHandler

- Purpose: Executes write operations to my_ratings.txt and read operations from both my_ratings.txt and movies.txt

4.2.3.1 Attribute Descriptions

1. Attribute: *movieCatalogPath*
   Type: Path
   Description: *Stores the relative path to movies.txt file in the file structure*
   Constraints*: N/A*
2. Attribute: *userRatingsPath*
   Type: Path

Description: *Stores the relative path to my_ratings.txt file in the file structure*
Constraints*: N/A*

4.2.3.2 Method Descriptions

1. Method: *updateExistingRating(int id, int rating, DateTime date)*
   Return Type: *void*
   Parameters: *id – the validated movie id that the user wants to rate; rating – the validated rating associated with the movie; date – the timestamp representing when the rating was decided*
   Return value: *N/A*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *userRatingPath*
   Methods called: *N/A*

   Processing logic:
   Loop through the lines within user_ratings.txt to find the line that holds the movie's previous rating. Once found, update the rating and timestamp using the passed in arguments

2. Method: *addNewRating(int id, int rating, DateTime date)*
   Return Type: *void*
   Parameters: *id – the validated movie id that the user wants to rate; rating – the validated rating associated with the movie; date – the timestamp representing when the rating was decided*
   Return value: *N/A*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *userRatingPath*
   Methods called: *N/A*

   Processing logic:
   Append a new line at the end of my_ratings.txt with the passed in movie id, rating, and timestamp

3. Method: *fetchMovieCatalog()*
   Return Type: *List<Movie>*
   Parameters: *N/A*
   Return value: *List of all movies and their associated data available for the user to view and rate*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *movieCatalogPath*
   Methods called: *N/A*

   Processing logic:
   Fetches all movie data from movies.txt by looping through each line and creating a new Movie object at each iteration. Lines with malformed data, missing fields, and invalid data types are skipped. Once each line is validated, a new Movie object is created and added to a collector list that is returned at the end of the method.

4. Method: *fetchUserRatings()*
   Return Type: *Map<Integer, Integer>*
   Parameters: *N/A*
   Return value: *Mapping from movie id to user rating. Represents all movies the user has watched so far and the rating value that was assigned*

Pre-condition: *N/A*
Post-condition: *N/A*
Attributes read/used: *userRatingPath*
Methods called: *N/A*

Processing logic:
Fetches all user rating data from my_ratings.txt by looping through each line and entering a key-value pair into a map representing the movie id -> user-rating

4.2.4 Class: RecommendationEngine

- Purpose: Analyze user ratings and generate movie recommendations for the user

4.2.4.1 Attribute Descriptions

1. Attribute: *movieCatalog*
   Type: *List<Movie>*
   Description: *Stores a list of all movies and their associated data read from movies.txt*
   Constraints*: Initialized to a non-zero size during object instantiation. Unchanged state after initialization*
2. Attribute: *fileHandler*
   Type: FileHandler
   Description: *Provides access to file handling functionality to initialize the movieCatalog field and call the appropriate method responsible for reading user ratings from my_ratings.txt.*
   Constraints*: non-null*

4.2.4.2 Method Descriptions

1. Method: *getMovieCatalog()*
   Return Type: *List<Movie>*
   Parameters: *N/A*
   Return value: *movieCatalog field*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *movieCatalog*
   Methods called: *N/A*

   Processing logic:
   Simple getter for movieCatalog field to be used ConsoleUI. displayExistingMoviesToUser() method.

2. Method: *generateMovieRecommendations()*
   Return Type: *List<Movie>*
   Parameters: *N/A*
   Return value: *Calculated list of movie recommendations tailored to user preferences based on previous user ratings stored in my_ratings.txt file.*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *movieCatalog, fileHandler*
   Methods called: *fileHandler.fetchUserRatings()*

Pseudocode:

```
userRatings ← fileManager.getUserRatings()
if userRatings is empty then
```

sort movieCatalog based on average rating and number of ratings
return movieCatalog

unwatchedMovies ← new List<Movie>()
for each movie in movieCatalog
if the movie id exists in userRatings, continue
Otherwise, add the movie to unwatchedMovies

highlyRatedMovies ← filter userRatings for movies with rating of 4 or higher
genreToCount ← create a mapping that maps genre to the count of highly rated movies

scoreToMovie ← new TreeMap<Integer, Integer>
for each movie in unwatchedMovies
calculate a similarity score based on genreToCount map created earlier
insert the score as the key and the movie id as the value in scoreToMovie

create a new list with the movie id's of the top 5 similarity scores in scoreToMovie map
return the list

4.2.5 Class: Movie

- Purpose: Stores the data associated with a movie record in the movies.txt file

4.2.5.1 Attribute Descriptions

1. Attribute: *id*
   Type: *int*
   Description: *Unique id associated with each movie in movies.txt*
2. Attribute: *title*
   Type: *String*
3. Attribute: *director*
   Type: *String*
4. Attribute: *year*
   Type: *String*
5. Attribute: *genres*
   Type: *List<Genre>*
6. Attribute: *averageRating*
   Type: *double*
7. Attribute: *numberOfRatings*
   Type: *int*

4.2.5.2 Method Descriptions

Aside from getters for each movie attribute:

1. Method: *createGenreList(String genres)*
   Return Type: *List<Genre>*
   Parameters: *A semicolon-delimited string of genre names read from movies.txt.*
   Return value: *A list of Genre enumeration values representing the genres associated with the movie.*
   Pre-condition: *N/A*
   Post-condition: *N/A*
   Attributes read/used: *genres field*
   Methods called: *N/A*

   Processing logic: This method initializes the genres field when constructing a new Movie instance. It parses the semicolon-delimited input string and converts each genre token into its

corresponding Genre enum value. The resulting list of Genre values is used to populate the genres attribute of the Movie.

5 Dynamic Model

Two UML sequence diagrams are included in this section to clarify the system's dynamic behavior. Notice that the FileHandler lifeline is independent of the lifelines for RatingManager and RecommendationEngine. This is intentional: the FileHandler instance is supplied to both classes through *dependency injection*. In other words, neither of these business-logic classes creates its own FileHandler; instead, a shared FileHandler instance is passed into their constructors.

Similarly, the lifelines of RatingManager and RecommendationEngine are independent of the ConsoleUI lifeline. ConsoleUI does not create these objects internally; rather, instances of RatingManager and RecommendationEngine are provided to ConsoleUI's constructor, following the same dependency-injection pattern.

5.1 Rating a Movie Scenario

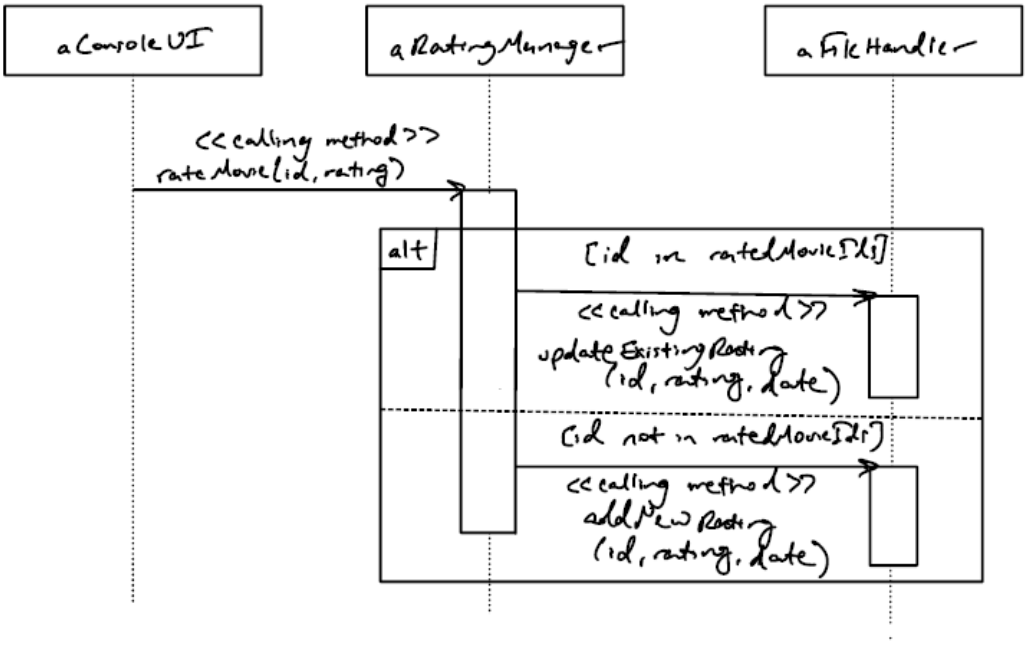


Figure 3: UML Sequence Diagram of Movie Rating Scenario

Figure 3 illustrates the system behavior when a user rates a movie. In this scenario, the ConsoleUI method responsible for prompting and parsing user input is promptUserToRateMovie(), which invokes the public rateMovie() method on the RatingManager. After validating the user's input, rateMovie() determines whether the specified movie ID has already been rated by checking for its presence in the ratedMovieIds hash set. If the ID is found, the existing rating is updated via updateExistingRating(); otherwise, a new rating is created using addNewRating().

5.2 Generating Movie Recommendations Scenario

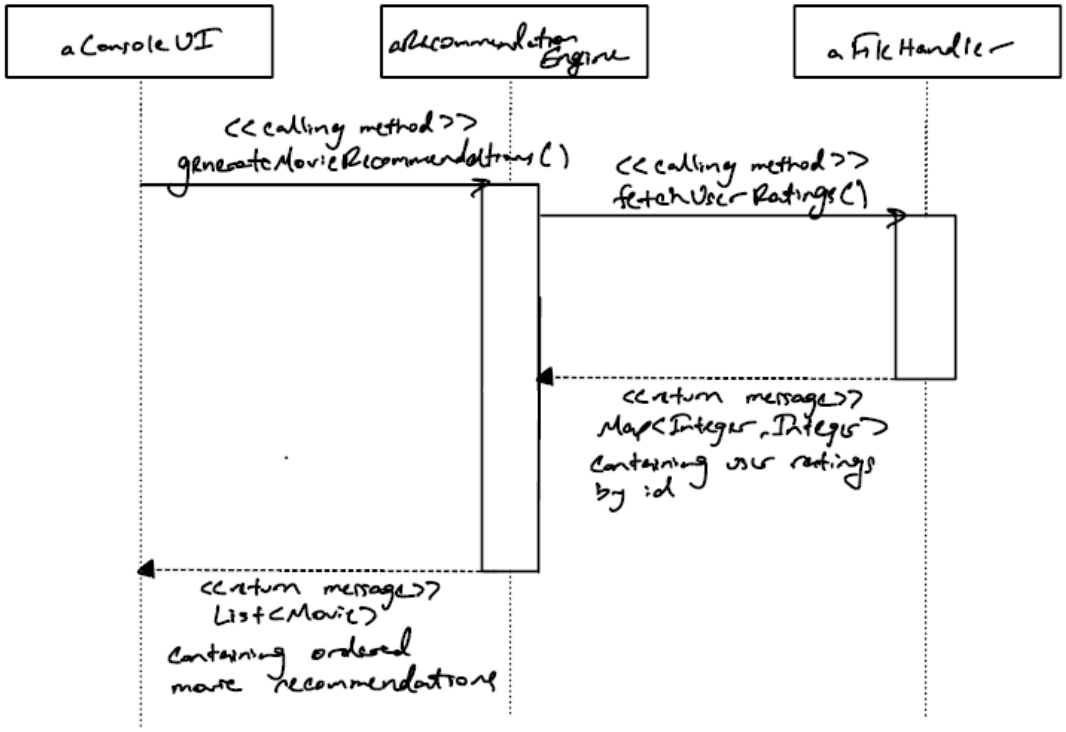


Figure 4: UML Sequence Diagram of Recommendation Generation Scenario

Figure 4 illustrates the system behavior when a user requests movie recommendations. In this scenario, the ConsoleUI method responsible for initiating the recommendation process is displayRecommendationsToUser(), which calls the public generateMovieRecommendations() method on the RecommendationEngine. Early in its execution, generateMovieRecommendations() retrieves the user's rating history by invoking fetchUserRatings() on the FileHandler, which returns a hash map mapping each watched movie to its user-assigned rating. Using this information, the recommendation algorithm produces a list of recommended movies and returns it to the ConsoleUI for display in the terminal.

6 Error Handling Strategy

6.1 Principles

- Crash proof: Upon any recoverable errors, the program will never terminate but instead handled.
- Effective error communication: The console UI will print concise messages in a predictable format always beginning with *'ERROR:'* for errors or '*OK'* for confirmation/successes.

6.2 Error Cases and Results

6.2.1 Inability to Start

Example: *movies.txt* missing, unreadable, corrupt, etc.

Result: Print cause, then exit gracefully as the program is unable to run without a functioning movie catalog.

6.2.2 Recoverable File

Example: *my_ratings.txt* file is inaccessible.
Result: Create a new *my_ratings.txt* file upon first rating. Otherwise, print '*ERROR:'* with cause and return to the menu.

6.2.3 Parsing

Example: Unmatching field type, incorrect date format, extra commas, etc.
Result: Skip line and show one diagnostic per improper line.

6.2.4 User Input

Example: Invalid *movie_id,* rating not between 1-5, Non-integer menu, etc.
Result: Display *'Error:'* with error cause and reprompting avoiding program crash.

6.2.5 Algorithmic Edge

Example: No user ratings
Result: Use popularity fallback (*avg_ratings, num_ratings*).

6.3 Responsibilities per layer

6.3.1 ConsoleUI

- Catches exceptions from services and prints user-friendly feedback.
- Upon user input errors, it will re-prompt the user.
- Program will only be exited if errors prevent program startup or the user terminates.

6.3.2 Business logic (RatingManager, RecommendationEngine)

- Ensures valid movie IDs.
- Ensures inputted ratings range from 1-5.
- Throws exception for bad input.
- Implements popularity feedback when no ratings exist.

6.3.3 FileHandler

- Read: Trim fields, perform type check, skip invalid lines and return appropriate diagnostics.
- Write: If absent, create *my_ratings.txt.* Otherwise, append to the file appropriately.

6.4 Standard Messages from CLI

6.4.1 Startup Inability

'ERROR: data/movies.txt not found or empty. This application cannot commence without a valid movie catelog.'

6.4.2 Invalid Menu Choice

‘ERROR: Enter one of the listed options (1, 2, 3, q).`

6.4.3 Invalid Movie ID

‘ERROR: Movie ID not found. Check Movie Catalog to see valid IDs.’

6.4.4 Invalid Rating

‘ERROR: Rating must be an integer between 1-5.`

6.4.5 Write Failure

‘ERROR: Could not save rating (write issue). Try again.`

6.5 Error Scenarios

6.5.1 movies.txt invalid

Print startup error and exit.

6.5.2 my_ratings.txt missing

Create file upon first rating, treat it as empty ratings.

6.5.3 Invalid lines in the catalog.

Skip invalid lines then return a diagnostic line including line number and reason.

6.5.4 Invalid user input

Displays *‘ERROR:’* + reason and then re-prompt user. Additionally, services perform secondary validation and throw ValidationException.

6.6 Methods of testing

- Test expected console outputs for each case.
- Use test CSV files with invalid lines to test diagnostic return.
- Confirm that invalid input never crashes the application.
- Confirm that startup is exited if catalog does not exist.
- Ensure that ratings file is created upon first save.

7 Non-functional Requirements

This section explains how our Movie Recommendation System design meets the non-functional requirements of the SRS. Since the system is very small and operates completely offline, most of these are addressed directly through simple design choices, Java built-ins, and the separation between the file-handling, rating management, and recommendation components. Below, I describe how our design satisfies each requirement area.

7.1 Performance Requirements

From the SRS, the primary performance expectations are for the system to load the movie catalog quickly, print out results to the terminal, and generate recommendations in less than one second. Our design addresses this by keeping all major operations lightweight:

- We load both movies.txt and my_ratings.txt once at startup using efficient Java I/O: BufferedReader.
- Parsed data is stored in memory using simple Java objects- Movie and RatingManager-that make lookup and filtering extremely fast.
- The step of recommendation relies on a simple genre-similarity calculation; hence, the algorithm runs in linear time with regard to the number of movies.

The design comfortably meets the timing requirements in the SRS because there is no networking or heavy computation within the system.

7.2 Safety Requirements

Our design prevents accidental damage to the user's filesystem by restricting all file operations to the local /data directory, exactly as stated in the SRS. We only ever read or write two files - movies.txt and my_ratings.txt - and we never attempt to access anything outside the program folder. To avoid data corruption:

- All file reads/writes use Java's try-with-resources so streams are always closed properly
- Updates to my_ratings.txt are written atomically by rewriting only the rating entry needed
- Parsing errors do not crash the system: we skip invalid lines and show an error message to the user.

This ensures that, even if the input files are partially broken or missing, the program behaves safely.

7.3 Security Requirements

Although the system is offline and single-user, we still considered basic security concerns listed in SRS. Our design avoids exposing unnecessary information:

- We never display system-level details in error messages
- The program doesn't open or send any network connections.
- User ratings remain local to my_ratings.txt and are not shared with anything outside the application.
- Only files inside /data can be accessed; all other paths are ignored.

These simple safeguards match the scale of the project, while keeping user data private.

7.4 Usability Requirements

Since the whole interface is text-based, we keep our design simple and predictable. The terminal UI shows:

- An easily comprehensible main menu
- numbered options
- a ">" input prompt
- consistent "ERROR:" and "OK:" messages

We tried to remove confusion by validating all inputs before doing anything else. For instance, when the user enters an invalid movie ID or rating, the system re-prompts instead of crashing or producing unclear behavior. Overall, the system is designed so that a beginner can use it with no prior instructions.

7.5 Maintainability Requirements

One of the major goals in our design was maintainability. We have divided the system into four logical components:

- FileHandler: responsible only for reading/writing text files
- RatingManager — manages user rating logic
- RecommendationEngine — calculates top-5 movies
- Movie and related models — store movie data.

This separation makes it easier to update code later. For instance, if we wanted to change how recommendations work or add new ranking rules, we only need to update the RecommendationEngine without touching file parsing or UI logic. Also, we use meaningful naming of classes and methods, keep logic modular, which supports easier debugging and testing.

7.6 Reliability Requirements

To enhance reliability, all major components handle errors gracefully. Our design ensures:

- Missing files (such as my_ratings.txt) don't cause crashes: that file is created automatically.
- The system is designed to safely skip malformed lines in the data files and continue its execution.
- Input validation prevents invalid states - like rating a movie that doesn't exist. –
- This application does not quit unless some genuinely unrecoverable error happens (ex: missing movies.txt at startup).

Because of this, even when the information entered by the user is imperfect, the system can still work.

7.7 Scalability Requirements

Even though this is a small academic project, our design tries to allow for future extension. For example:

- Genres are stored in a list, so adding new similarity rules is easy.
- The recommendation engine works with the lists of Movie objects, thus enabling more advanced algorithms to substitute the current one without rewriting other components.
- The file format is a simple CSV, but the structure of the system would allow moving to a more structured format later in the process.

This means that the system can grow beyond the course project if future teams wanted to extend it.

7.8 Portability Requirements

Since everything is written in plain Java using only standard libraries, the system will run on any machine with Java 17+. No external dependencies, toolkits, or frameworks are used. This is precisely what the SRS said it should be, making the system easy to run on Windows, macOS, or Linux.

7.9 Future Evolution

Although the system is simple now, the design allows for future improvements. Potential extensions, such as improving the algorithm, adding search capabilities, migrating to a GUI, or allowing for extended catalog formats, can be achieved during the extension of existing classes without rewriting the entire application. This flexibility is intentional and follows the long-term evolution described in the course project.